

第3章 动态规划

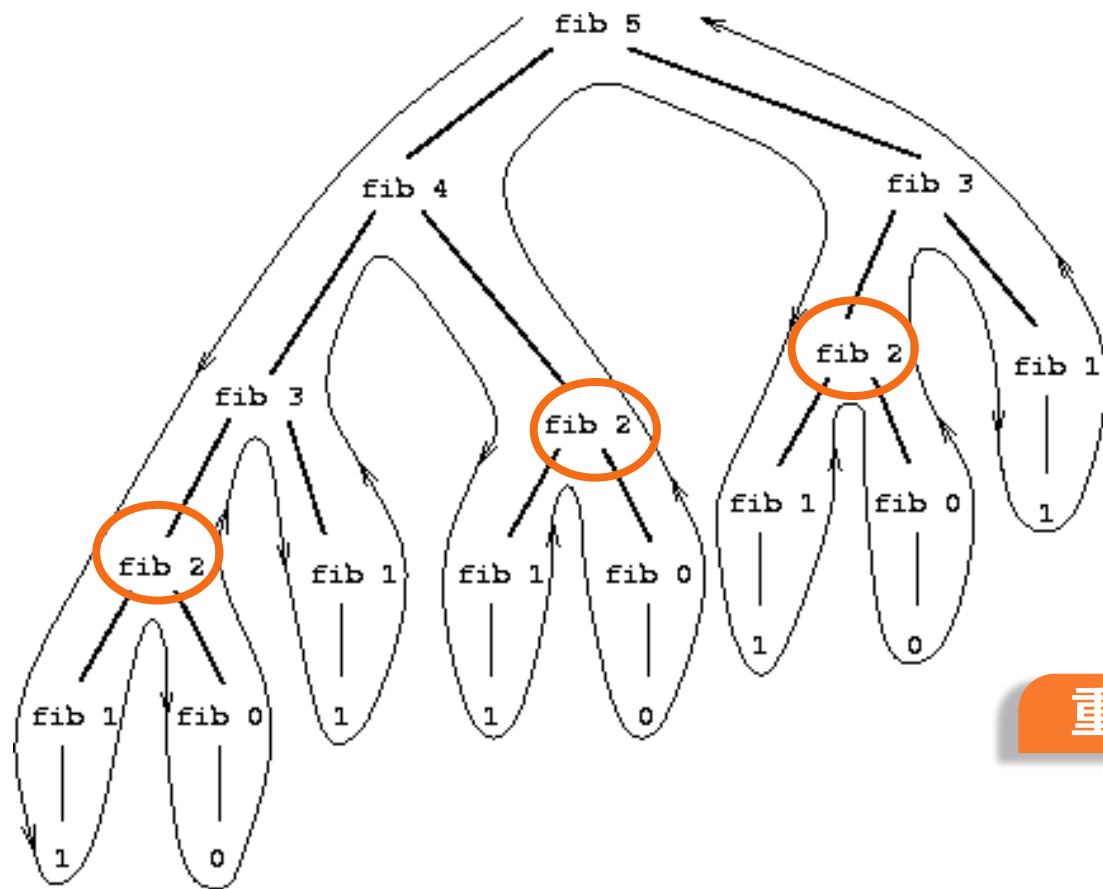
引子

$$F(n) = \begin{cases} 1 & n = 0, 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

分治（递归）算法:

```
int fib(int n){  
    if ((n==0) || (n==1))  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

引子



重复子问题!!

引子

算法复杂度是 $O(n)$

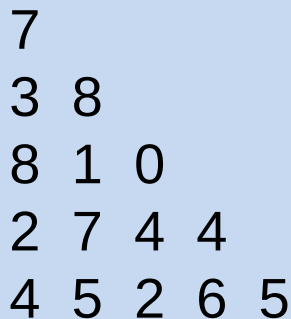
```
long fibonacci(int n)
{
    long* fib = new long[n + 1];
    fib[0] = 1; fib[1] = 1;
    for (int i = 2; i <= n; i++)
        fib[i] = fib[i - 1] + fib[i - 2];
    return fib[n];
}
```

	F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	...
fib	1	1	2	3	5	8	...

引子

数字三角路径问题:

给定等腰直角数字三角形，请编一个程序计算从顶至底的某个位置的一条路径，使该路径所经过的数字的总和**最大**。假设每一步可延直线向下或右斜线向下走。



```
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5
```

- $a[i][j]$ 为第*i*行第*j*列的数字
- $opt[i][j]$ 记录从顶部到第*i*行第*j*列位置时获得的**最大总和**

边界条件

- 1.顶部: $opt[0][0] = a[0][0]$;
 2. $j=0$ $opt[i][0] = a[i][0] + opt[i-1][0]$ //最左侧
 3. $i=j$ $opt[i][i] = opt[i-1][i-1] + a[i][i]$ //最右侧
- else

$$opt[i][j] = a[i][j] + \max(opt[i-1][j], opt[i-1][j-1])$$

结果: $\max\{opt[N-1][j], 0 \leq j \leq N-1\}$

动态规划算法总体思想

- **动态规划算法与分治法类似**，其基本思想也是**将待求解问题分解成若干个子问题**；（即把原问题拆分成若干个规模更小、结构相同的子问题。）
- 但经分解得到的子问题往往**不是互相独立的**。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。
（即记住每个子问题的最优解（用数组 / 表格存起来），避免重复计算。）
- 能够**保存已解决的子问题的答案**，在需要时再找出已求得的答案，避免大量重复计算，从而得到多项式时间算法。
（即通过子问题的最优解，逐步推导出原问题的最优解。）

背景

- **动态规划 (Dynamic Programming)** 是运筹学的一个分支，20世纪50年代初美国数学家R.E.Bellman等人在研究**多阶段决策过程**的优化问题时，提出了著名的**最优性原理**，把多阶段过程转化为一系列单阶段问题，逐个求解，创立了解决这类过程优化问题的新方法——**动态规划**。
- **多阶段决策问题**：求解的问题可以划分为一系列相互联系阶段，在每个阶段都需要做出决策，且一个阶段决策的选择会影响下一个阶段的决策，从而影响整个过程的**活动路线**，求解的目标是**选择各个阶段的决策使得整个过程达到最优**。
- 动态规划主要用于求解**以时间划分阶段的动态过程的优化问题**，但是一些与时间无关的静态规划（如线性规划、非线性规划），可以**人为地引进时间因素**，把它视为多阶段决策过程，也可以用动态规划方法方便地求解。

基本概念

- **状态和状态表示**: 表示每个阶段开始时, 问题或系统所处的客观状况。状态既是该阶段的某个起点, 又是前一个阶段的某个终点。通常一个阶段有若干个状态, 每个状态用多元组 $S(p_1, p_2, \dots, p_k)$ 表示, 值域表示状态值, 或者对应子问题的答案。
- **状态转移**: 在动态规划的多阶段决策过程中, 决策序列对应状态序列, 每一次决策对应一次状态转移, 状态转移用递推方程来描述, 称之为**状态转移方程**。
- **状态无后效性**: 如果某个阶段状态给定后, 则该阶段以后过程的发展不受该阶段以前各阶段状态的影响, 也就是说状态具有**马尔科夫性**。
- **Bellman最优性原理**: 求解问题的一个最优策略序列的子策略序列总是最优的, 则称该问题满足最优性原理。

矩阵连乘问题

问题描述: 对于 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$, 其中 A_i 与 A_{i+1} 可乘, 给定一个计算次序, 使得连乘的计算量最小。

(注: A 为 $p \times q$ 矩阵, B 为 $q \times r$ 矩阵, 则 $C = AB$ 为 $p \times r$ 矩阵, 且需要 $p \times q \times r$ 次数乘)

矩阵连乘问题

首先：考察两个矩阵相乘的情形： $C=AB$ 。如果矩阵A, B分别是 $p \times r$ 和 $r \times q$ 矩阵, 则它们的乘积C将是 $p \times q$ 矩阵, 其 (i, j) 元素为:

$$c_{ij} = \sum_{k=1}^r a_{ik}b_{kj}$$

$i = 1, \dots, p, j = 1, \dots, q$, 因而AB所用的数乘次数是 $p \times r \times q$ 。

如果有至少3个以上的矩阵连乘, 则涉及到乘积次序问题, 即**加括号方法**。例如3个矩阵连乘的加括号方法有两种: **$((A_1A_2)A_3)$** 和 **$(A_1(A_2A_3))$** 。设 A_1, A_2, A_3 分别是 $p_0 \times p_1, p_1 \times p_2, p_2 \times p_3$ 矩阵, 则以上两种乘法次序所用的数乘次数分别为:

$p_0p_1p_2 + p_0p_1p_3$ 和 $p_0p_1p_3 + p_1p_2p_3$ 。如果 $p_0=10, p_1=100, p_2=5, p_3=50$, 则两种乘法所用的数乘次分别为: **7500** 和 **750000**。

任意给定 n 个可乘的数字矩阵 $A_1, A_2, \dots, A_n$, 以及矩阵的维度 $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, 求解给定矩阵链的**最优计算次序**使得所需要的数乘次数最少。

完全加括号

矩阵连乘计算次序可以用**加括号的方式**来确定。特别的，**完全加括号**的矩阵连乘积可**递归地定义**为：

- 单个矩阵是完全加括号的
- 矩阵连乘积 A 是完全加括号的，则 A 可示为2个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A = (BC)$

给定4个矩阵 $A(50 \times 10)$ ， $B(10 \times 40)$ ， $C(40 \times 30)$ ， $D(30 \times 5)$ ，则这4个矩阵的连乘对应5种完全加括号方案：

$$\begin{array}{ccc}(A((BC)D)) & (A(B(CD))) & ((AB)(CD)) \\ (((AB)C)D) & ((A(BC))D) & \end{array}$$

16000, 10500, 36000, 87500, 34500

问题分析

给定 n 个矩阵 $A_1, \dots, A_n$, 其中第 i 个矩阵的维度为 $p_{i-1} \times p_i$, 以及它们的一个完全加括号方案:

$$((A_1(A_2 \cdots A_k)) ((A_{k+1}A_{k+2}) \cdots A_n)) \quad a$$

该完全加括号方案所需的数乘次数包括**三部分**:

- ① $(A_1(A_2 \cdots A_k))$ b
- ② $((A_{k+1}A_{k+2}) \cdots A_n)$ c
- ③ 上述两个结果矩阵相乘所需要的数乘次数 $p_0 p_k p_n$

$$a = b + c + p_0 p_k p_n$$

如果 a 是**最优值**, b 和 c 呢?

最优子结构性质

假设 n 个矩阵连乘的**最优加括号方案**为 $((A_1 \cdots A_k)(A_{k+1} \cdots A_n))$ (省略了 $(A_1 \cdots A_k)$ 和 $(A_{k+1} \cdots A_n)$ 内部的括号), 则加括号方案 $(A_1 \cdots A_k)$ 是子矩阵链 $A_1 \cdots A_k$ 的**最优方案**, $(A_{k+1} \cdots A_n)$ 是子矩阵链 $A_{k+1} \cdots A_n$ 的**最优方案**。

证明 (反证法) :

构建**原问题最优解**与**子问题最优解**之间的关系

原问题的最优加括号方案假设为 $((A_1(A_2 \cdots A_k))((A_{k+1}A_{k+2}) \cdots A_n))$, 如果 $(A_1(A_2 \cdots A_k))$ 不是 $A_1 \cdots A_k$ 的最优方案, 则 $A_1 \cdots A_k$ 至少存在一个加括号方案, 其数乘次数 b' 少于 b , 该方案记为 $((A_1A_2) \cdots A_k)$ 。

构造原问题的另外一个加括号方案: $((A_1A_2) \cdots A_k)((A_{k+1}A_{k+2}) \cdots A_n)$

$$\begin{aligned} a' &= b' + c + p_0 p_k p_n \\ a &= b + c + p_0 p_k p_n \\ b' &< b \end{aligned}$$

$$a' < a$$

状态表示和递推方程

- 状态表示本质上是**子问题的参数化描述**：每一个**子矩阵链** $A_i \cdots A_j$ 对应一个**子问题**，它由开始矩阵和结束矩阵的**下标**决定，记为 $A[i:j]$ 。
- $A[i:j]$ 的最优计算次序对应的乘法次数表示为 $m(i, j)$, $1 \leq i, j \leq n$,
- **原问题**的最优值为 $m(1, n)$ 。

1) $i = j$, $m(i, i) = 0$, $1 \leq i \leq n$, 因为一个矩阵无乘法

2) $i < j$, 且 $((A_i \cdots A_k)(A_{k+1} \cdots A_j))$ 为最优加括号方案, 则

$$m(i, j) = m(i, k) + m(k+1, j) + p_{i-1}p_kp_j$$

$$\min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\}$$

K 怎么确定?

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$

状态表示和状态转移方程

□ 状态:

子矩阵链 $A_i \cdots A_j$, 由开始矩阵和结束矩阵的**下标**决定, 记为 $A[i:j]$

□ 状态表示:

$A[i:j]$ 的最优计算次序对应的乘法次数表示为 $m(i, j)$, $1 \leq i, j \leq n$

□ 状态转移方程:

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$

子问题个数和求解顺序

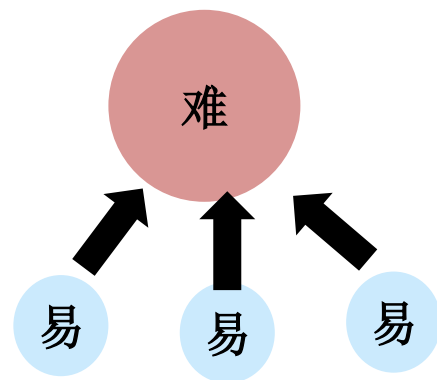
- ◆ 任意**有序对** $(i, j), 1 \leq i \leq j \leq n$ 对应于一个**子问题** $A(i, j)$, 则子问题的个数最多有:

1) $A(i, i): n$

2) $A(i, j): \binom{n}{2}$

- ◆ **自底向上**的顺序进行求解, 或者说**从易至难**的顺序求解各个子问题。

- ◆ 初始化, 令 $m(i, i) = 0, i = 1, 2, \dots, n$, 对应于状态矩阵 m 的对角线;
- ◆ 计算所有两个矩阵 $A_i A_{i+1}$ 相乘时的最优值;
- ◆ 依次计算:
 - 3个矩阵相乘
 -
 - $n-1$ 个矩阵相乘
 - n 个矩阵相乘



计算最优值示例

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

i \ j	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$

$$m(2, 5) = \min \begin{cases} m(2, 2) + m(3, 5) + p_1p_2p_5 = 13000 \\ m(2, 3) + m(4, 5) + p_1p_3p_5 = 7125 \\ m(2, 4) + m(5, 5) + p_1p_4p_5 = 11375 \end{cases}$$

计算最优值顺序

$i \backslash j$		1	2	3	4	5	6
1	1	0	15750	7875	9375	11875	15125
2	2		0	2625	4375	7125	10500
3	3			0	750	2500	5375
4	4				0	1000	3500
5	5					0	5000
6	6						0

算法实现

```
//自底向上地计算最优值，结果保存在全局变量memoTable和bestK中
long MatrixChain(int matrixNum) {
    int i,j,len,k;
    for(i=1; i<=matrixNum; i++) //单个矩阵的情形，定义数乘次数为0
        memoTable[i][i] = 0;
    for(len=2; len<=matrixNum; len++){ //计算长度为len的矩阵链最优值
        for(i=1; i<=matrixNum-len+1; i++){ //矩阵链的开始矩阵下标
            j = i+len-1; //矩阵链的结束矩阵下标
            memoTable[i][j] = 1000000000; //预定义的一个充分大数
            for(k=i; k<j; k++){ //枚举划分位置
                long ans = memoTable[i][k] + memoTable[k+1][j] +
                    dim[i-1]*dim[k]*dim[j];
                if (ans < memoTable[i][j]){ //更新最优信息
                    bestK[i][j] = k;
                    memoTable[i][j] = ans;
                }
            }
        } //end of for k
    } //end of for i
} //end of for len
return memoTable[1][matrixNum];
}
```

算法的计算时间上界为 $O(n^3)$

算法所占用的空间显然为 $O(n^2)$

构造最优解

$m(i, j)$ 只是计算了最优值，但是**没有**给出**最优解**，也就是说还不知道**完全加括号方案**？

$s(i, j) = k$ 表示计算矩阵链 $A[i:j]$ 的最佳方式应该放在矩阵 A_k 和 A_{k+1} 之间断开，即最优加括号方式为 $((A[i:k])(A[k+1:j]))$ 。

$s(1, n)$ 可以推导出**最外层括号**：

$A[1:n]$ 的最优加括号方式为 $((A[1:s(1, n)])(A[s(1, n) + 1:n]))$ ；

$s(1, s(1, n))$ 可以推导出 $A[1:s(1, n)]$ 的**最外层括号**：

$A[1:k]$ 的最优加括号方式为 $((A[1:s(1, k)])(A[s(1, k) + 1:k]))$ ，
其中 $k = s(1, n)$

... 照此递推，**自顶向下**构造矩阵链的完全加括号方案。

实例：计算矩阵连乘积 $A_1A_2A_3A_4A_5A_6$ ，维数为：

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

i\j	1	2	3	4	5	6
1	0	15750 s=1	7875 s=1	9375 s=3	11875 s=3	15125 s=3
2		0	2625 s=2	4375 s=3	7125 s=3	10500 s=3
3			0	750 s=3	2500 s=3	5375 s=3
4				0	1000 s=4	3500 s=5
5					0	5000 s=5
6						0

$(A_1A_2A_3) (A_4A_5A_6)$

$(A_1 (A_2A_3)) (A_4A_5A_6)$

$(A_1 (A_2A_3)) ((A_4A_5) A_6)$

最优解为：

$(A_1 (A_2A_3)) ((A_4A_5) A_6)$

状态表示和状态转移方程

□ 状态:

子矩阵链 $A_i \cdots A_j$, 由开始矩阵和结束矩阵的**下标**决定, 记为 $A[i:j]$

□ 状态表示:

$A[i:j]$ 的最优计算次序对应的乘法次数表示为 $m(i, j)$, $1 \leq i, j \leq n$

□ 状态转移方程:

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k + 1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$

递归方法

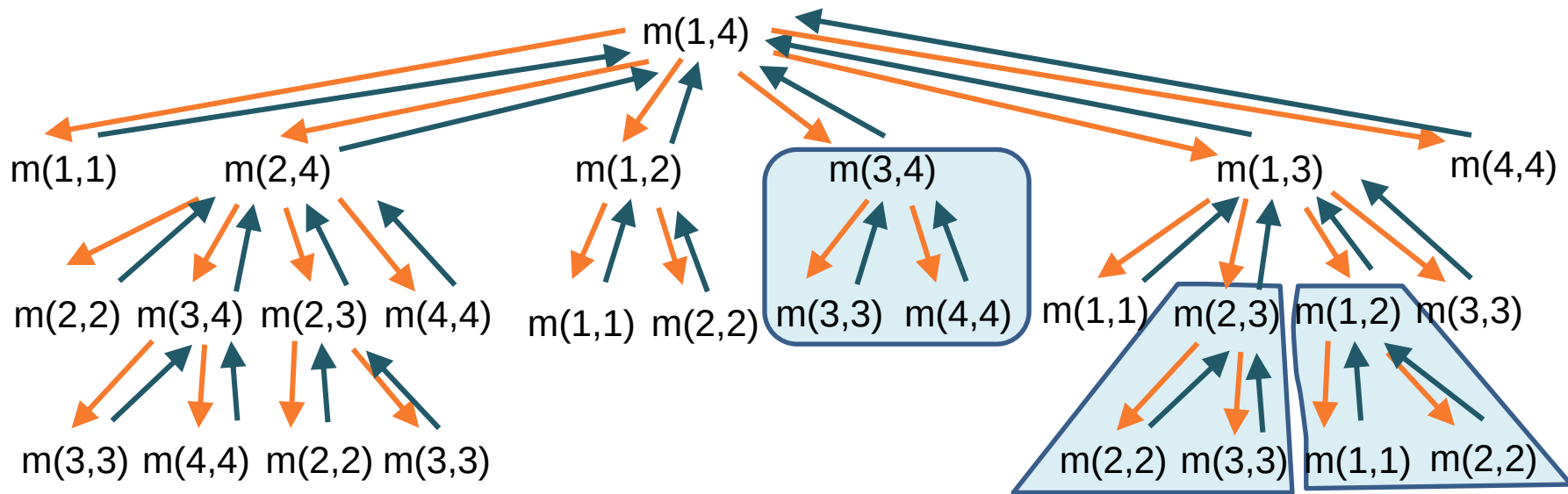
```
long MatrixChain(int i, int j){  
    if (i == j) { //单个矩阵的情形  
        return 0;  
    }  
    long ans, minV = 1000000000; //预定义的一个充分大数  
  
    for (int k=i; k<j; k++) {  
  
        ans = MatrixChain(i,k) + MatrixChain(k+1,j)  
            + dim[i-1]*dim[k]*dim[j]; //递归计算  
        if (ans < minV) {  
            minV = ans;  
        }  
    }  
    return minV; }
```

指数级别时间复杂度!

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$

递归过程

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$



备忘录方法

- ◆ 备忘录方法**用表格保存**已解决的子问题的**答案**，在下次需要解决此问题时，只要从表格中**提取答案**即可，而不需要重新计算；如果没有求解，则递归调用求解过程进行计算。
- ◆ 备忘录方法的控制结构与直接递归方法的控制结构相同，**自顶向下**的处理，程序简单；区别在于备忘录方法**为每个求解过的子问题建立了备忘录**以备需要时查看，避免了相同子问题的重复求解。

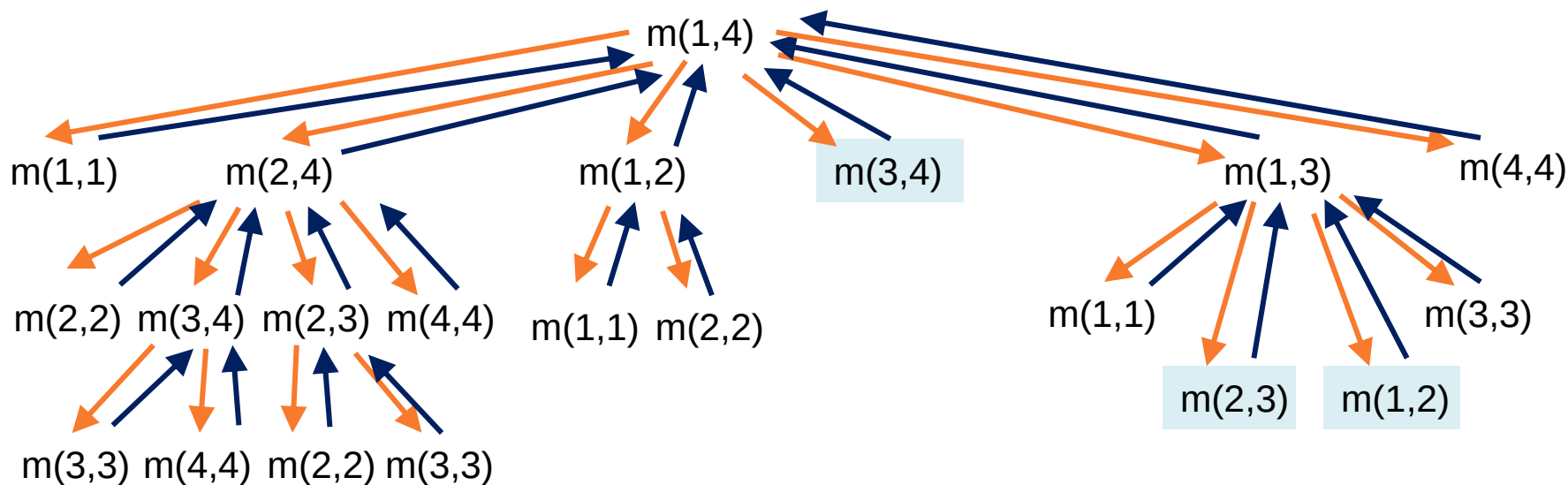
备忘录方法

//调用前用 `memset(memoTable,-1,sizeof(memoTable))`初始化备忘录表为-1

```
long MatrixChainMemo(int i,int j){  
    if (memoTable[i][j] != -1)  
        return memoTable[i][j]; //备忘录表中有答案  
    if (i == j) { //单个矩阵的情形  
        memoTable[i][j] = 0;  
        return 0;  
    }  
    long ans, minV = 1000000000; //预定义的一个充分大数  
    for (int k=i; k<j; k++) {  
        ans = MatrixChainMemo(i,k)+MatrixChainMemo(k+1,j)  
            +dim[i-1]*dim[k]*dim[j]; //递归计算  
        if (ans < minV) {  
            bestK[i][j]=k;  
            minV = ans;  
        }  
    }  
    memoTable[i][j] = minV;  
    return minV;  
}
```

递归过程

$$m(i, j) = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\} & i < j \end{cases}$$



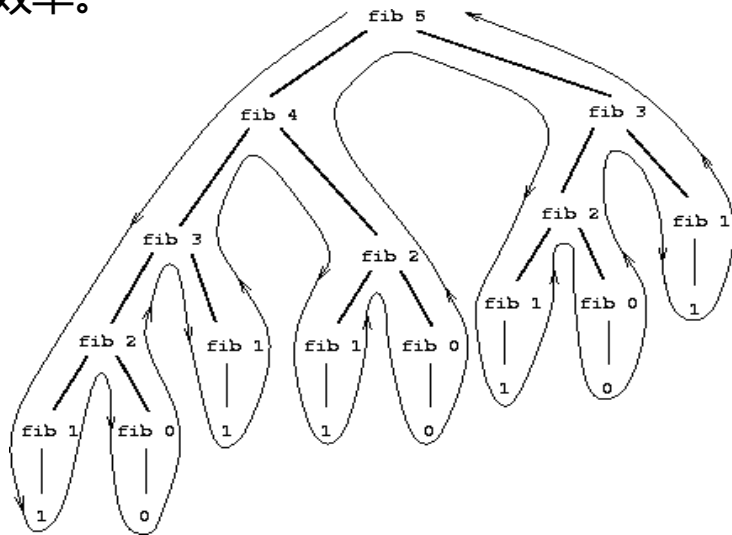
基本要素 – 最优子结构

- ◆ **最优子结构性质**，通俗地讲就是**问题的最优解包含其子问题的最优解**。也就是说，如果把问题的最优解分解（比如划分为两个或者多个部分，或者删除第一个或者最后一个分量），得到一个子解，那么这个**子解**是其**相应子问题的最优解**。
- ◆ 最优子结构性质**隐含了问题最优解和子问题最优解之间的一种递推关系**。它是动态规划的基础，递推方程是最优子结构性质的体现。
- ◆ 在分析问题的最优子结构性质时，人们一般采用**反证法**：首先**假设**由问题最优解S导出的子问题的解不是最优的，然后再推导在这个假设下可**构造**出比S更好的解 S' ，从而**得到矛盾**。

注意：同一个问题可以有多种方式刻画它的最优子结构

基本要素 – 重叠子问题

- ◆ 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为**子问题的重叠性质**。
- ◆ 动态规划算法，对**每一个子问题只解一次**，而后将其解保存在一个表格中，当**再次**需要解此子问题时，只是简单地用常数时间**查看结果**。
- ◆ 通常不同的**子问题数目**随问题的大小呈**多项式**增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。



判断题：

- 动态规划算法适用于所有具有最优子结构性质的问题。
- 答：**错误**。最优子结构是动态规划算法的必要条件之一，但并不是充分条件。除了最优子结构，问题还必须满足重叠子问题的性质，即子问题需要被重复计算。如果问题没有重叠子问题，动态规划的优势（通过存储子问题解避免重复计算）就无法体现。

动态规划算法的设计步骤:

1. 分析最优解的性质，并刻画其**最优子结构**特征;
2. 确定**状态表示** $S(x_1, x_2, \dots)$ 和**状态递推方程**，递归地定义最优值;
3. 确定**状态转移顺序**，以自底向上的方式计算出最优值;
4. 根据计算最优值时得到的信息，**构造最优解**。

动态规划能够有效求解问题的两个必要前提（核心性质）是？

- ☐ A 贪心选择性质和最优子结构性质
- ☒ B 最优子结构性质和重叠子问题性质
- ☐ C 无后效性和局部最优性
- ☐ D 状态可枚举性和决策独立性

提交

最优子结构性性质是指？

- ☐ A 问题的最优解包含了部分子问题的最优解
- ☐ B 问题的最优解不包含其子问题的最优解
- ☒ C 问题的最优解包含了其子问题的最优解
- ☐ D 以上都不对

提交

单选题 2分



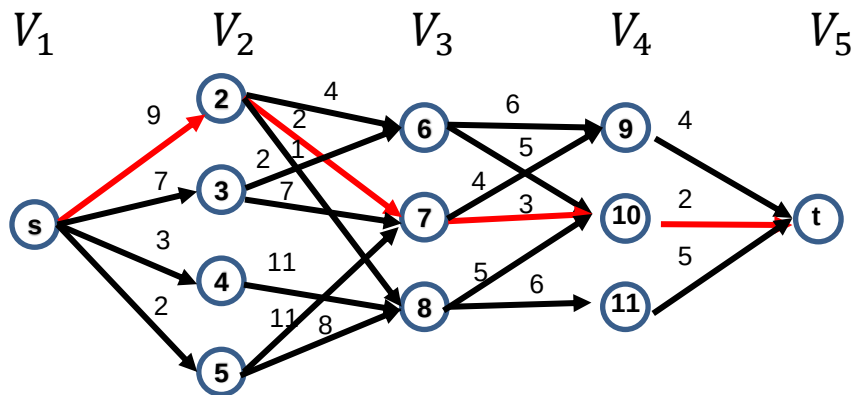
已知矩阵链 $A_1(2 \times 3)$ 、 $A_2(3 \times 4)$ 、 $A_3(4 \times 5)$ ，采用动态规划求解该矩阵连乘的最小标量乘法次数，其状态转移中涉及的核心计算式为（）（注： $m[i][j]$ 表示 $A_i \sim A_j$ 连乘的最小标量乘法次数）

- ☐ A $m[1][3] = \min\{m[1][1] + m[2][3] + 2 \times 3 \times 5, m[1][2] + m[3][3] + 2 \times 4 \times 5\}$
- ☒ B $m[1][3] = \min\{m[1][1] + m[2][3] + 2 \times 3 \times 5, m[1][2] + m[3][3] + 2 \times 3 \times 5\}$
- ☐ C $m[1][3] = m[1][2] + m[2][3] + 3 \times 4 \times 5$
- ☐ D $m[1][3] = \min\{m[1][1] + m[2][3] + 3 \times 4 \times 5, m[1][2] + m[3][3] + 2 \times 3 \times 4\}$

提交

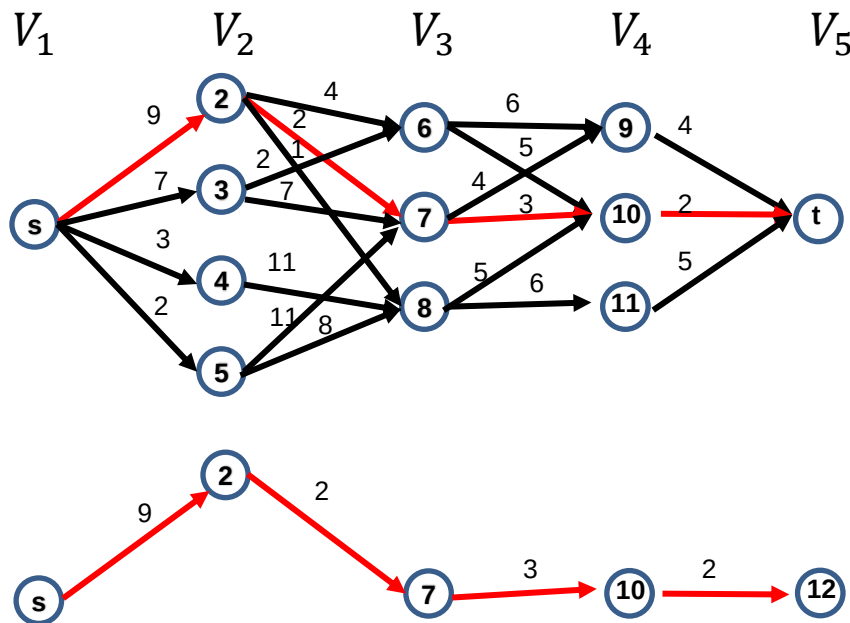
多段图最短路径

问题描述： 设 $G = (V, E)$ 是一个赋权有向图，其顶点集 V 被划分成 k ($k > 2$) 个不相交的子集 V_i ($1 \leq i \leq k$)，其中 V_1 和 V_k 分别只有一个顶点 s (称为源) 和一个顶点 t (称为汇)，所有的边 (u, v) 的始点和终点都在相邻的两个子集 V_i 和 V_{i+1} 中： $u \in V_i, v \in V_{i+1}$ ，且边 (u, v) 有一个正权重，记为 $w(u, v)$ 。请设计一个算法，求解从源 s 到汇 t 的权重之和**最小的路径**。



最优子结构性质

给定多段图 $G = (V, E)$, 假设 $s \rightarrow v(2, p_2) \rightarrow \dots \rightarrow v(k-1, p_{k-1}) \rightarrow t$ 是一条由 s 到 t 的最短路径, 其中 $v(k-1, p_{k-1})$ 表示第 $k-1$ 段顶点集中的顶点 p , 则 $s \rightarrow v(2, p_2) \rightarrow \dots \rightarrow v(k-1, p_{k-1})$ 是子图 $G' = (V', E')$ 的最短路径, 其中 $V' = (V - V_k - V_{k+1}) \cup \{v(k-1, p_{k-1})\}$, E' 为 E 只包含 V' 中顶点的边集, G' 的汇点为顶点 $v(k-1, p_{k-1})$ 。



最优子结构性质

给定多段图 $G = (V, E)$ ，假设 $s \rightarrow v(2, p_2) \rightarrow \dots \rightarrow v(k-1, p_{k-1}) \rightarrow t$ 是一条由 s 到 t 的最短路径，其中 $v(k-1, p_{k-1})$ 表示第 $k-1$ 段顶点集中的顶点 p ，则 $s \rightarrow v(2, p_2) \rightarrow \dots \rightarrow v(k-1, p_{k-1})$ 是子图 $G' = (V', E')$ 的最短路径，其中 $V' = (V - V_k - V_{k+1}) \cup \{v(k-1, p_{k-1})\}$ ， E' 为 E 只包含 V' 中顶点的边集， G' 的汇点为顶点 $v(k-1, p_{k-1})$ 。

与前提矛盾！

证明（反证法）： $dis_q + w(v(k-1, p_{k-1}), t) < dis_p + w(v(k-1, p_{k-1}), t)$

如果 $s \rightarrow v(2, p_2) \rightarrow \dots \rightarrow v(k-2, p_{k-2}) \rightarrow v(k-1, p_{k-1})$ 不是子图 G' 的最短路径；
则子图 G' 存在另外一条路径 $s \rightarrow v(2, q_2) \rightarrow \dots \rightarrow v(k-2, q_{k-2}) \rightarrow v(k-1, p_{k-1})$ ，其路径长度**更小**，如果两者的路径长度分别记为 dis_p 和 dis_q ，则有 $dis_p > dis_q$ 。

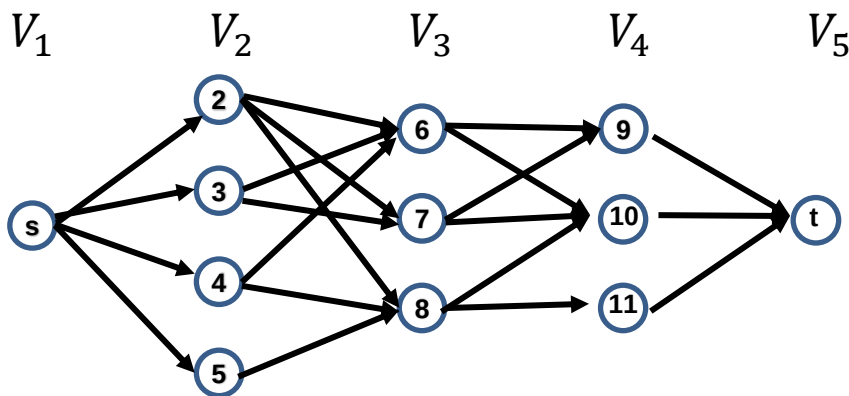
构造原图 G 的另外一条路径： $s \rightarrow v(2, q_2) \rightarrow \dots \rightarrow v(k-2, q_{k-2}) \rightarrow v(k-1, p_{k-1}) \rightarrow t$

新路径长度： $dis_q + w(v(k-1, p_{k-1}), t)$

最优解长度： $dis_p + w(v(k-1, p_{k-1}), t)$

状态表示

包含源结点 s 的子图 都可以认为是一个子问题，子图的源是固定的，汇是变化的



子问题实例

状态表示

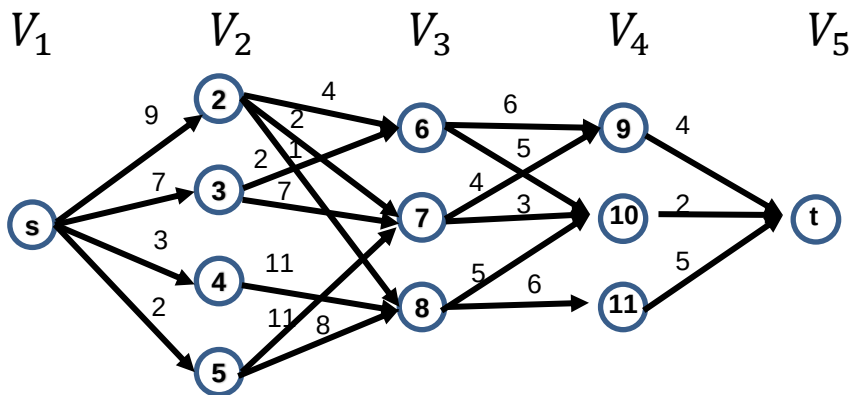
包含源结点 s 的子图对应一个子问题，子图的源是固定的，汇是变化的。因此，确定了汇的位置，则能确定一个子图。汇的位置包括两个参数：

1. 段的序号
2. 汇顶点在段中的序号

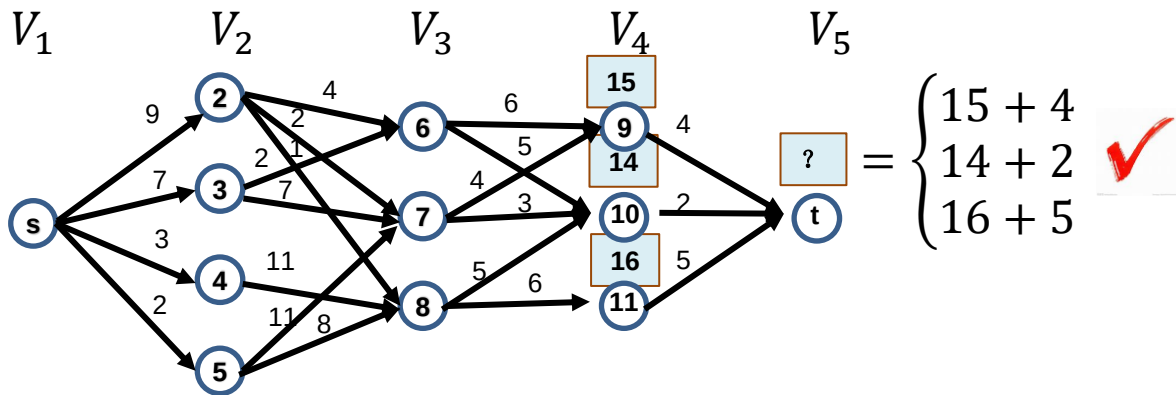
$W(i, p)$ 表示从源 s 到 $v(i, p)$ 的最短路径长度

◆ $i(1 \leq i \leq k)$ 表示段的序号

◆ $p(1 \leq p \leq n_i)$ 表示第 i 段顶点集中的顶点序号



状态转移方程



$$W(i, p) = \begin{cases} 0, & i = 1 \\ \min_{q \in V_{i-1}, (q, p) \in E} \{W(i-1, q) + w(q, p)\}, & i > 1 \end{cases}$$

状态表示和状态转移方程

□ 状态:

从**源结点**到任意**中间结点**（子图的汇结点）的最短路径，由当前**汇结点**确定

□ 状态表示:

$W[i, p]$ 表示从源 s 到中间结点 $v(i, p)$ 的最短路径长度

□ 状态转移方程:

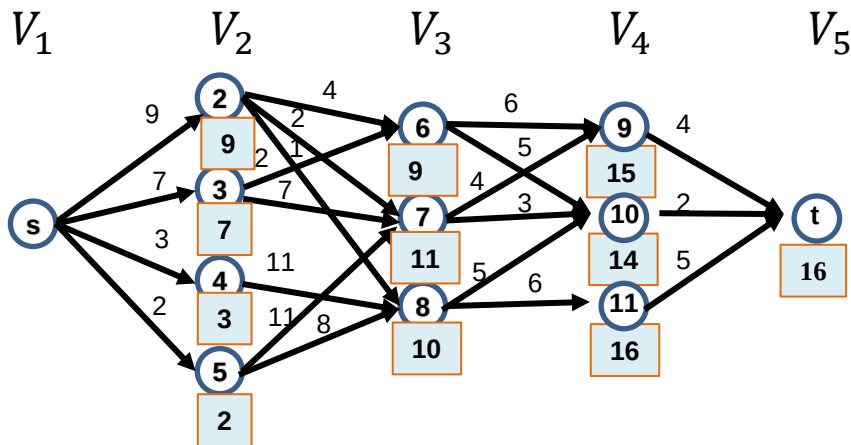
$$W(i, p) = \begin{cases} 0, & i = 1 \\ \min_{q \in V_{i-1}, (q, p) \in E} \{W(i-1, q) + w(q, p)\}, & i > 1 \end{cases}$$

最优值求解

◆ 子问题的数目等于图G中**顶点的个数**

◆ 采用**自底向上（由易至难）**的方法求最优值：

- 最开始求解源s到第2段顶点集中每一个顶点的最短路径，**最简单**。
- 然后求解s到第3段顶点集中的每一个顶点的最优值
- **依此循环**，直至求解s到t的最短路径值。



```

#include <math.h>
#include<stdio.h>
#include<memory.h>
#define MaxStage 100
int minRoad[MaxStage][MaxStage];
int MultiStageGraph(int,int*);
int main(){
    int ni[MaxStage],k, i;
    while(scanf("%d",&k),k!=-1) {
        for(i=0; i<k; i++)
            scanf("%d",&ni[i]);

        printf("%d\n",MultiStageGraph(k,ni));
    }
    return 0;
}

```

主程序

算法实现

```

int MultiStageGraph(int stageNum,int *numPerStage) {
    int i,q,p,weight,temp;
    memset(minRoad,0x3f,sizeof(minRoad)); //初始化为一个充分大的数
    for(p=0; p<numPerStage[0]; p++) //初始化源顶点层
        minRoad[0][p]=0;
    for(i=0; i<stageNum-1; i++)//按段计算，终止于汇顶点的前一段
        for(q=0; q<numPerStage[i]; q++) //遍历第i段顶点
            for(p=0; p<numPerStage[i+1]; p++) { //遍历第i+1段顶点
                scanf("%d",&weight); //读取边(q,p)的权重w(q,p)
                if(weight!=-1){ //存在边(q,p)
                    temp = minRoad[i][q] + weight;
                    if(temp < minRoad[i+1][p]) //发现s到p的更短路径
                        minRoad[i+1][p] = temp;
                }
            } // end for p
    return minRoad[stageNum-1][0];
}

```

最长公共子序列

问题描述：若给定序列 $X = \{x_1, x_2, \dots, x_m\}$ 、 $Z = \{z_1, z_2, \dots, z_k\}$ ，若 Z 是 X 的子序列，当且仅当存在一个严递增下标序列 $\{i_1, i_2, \dots, i_k\}$ ，使得对于所有 $j = 1, 2, \dots, k$ 有： $z_j = x_{i_j}$ 。例如，序列 $Z = \{B, C, D, B\}$ 是序列 $X = \{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。

给定序列 X 和 Y ，如果序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的**公共子序列**。例如 $X = \{A, B, C, B, D, A, B\}$ ， $Y = \{C, B, C, E, D, B\}$ ，则 $\{B, C, D, B\}$ 是 X 和 Y 的公共子序列，当然 $\{B, C\}$ ， $\{B, C, D\}$ 等也都是 X 和 Y 的公共子序列。在两个序列的所有子序列中，包含元素最多的序列称之为**最长公共子序列**。

任意给定两个字符序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ ，设计算法求解它们的最长公共子序列。

- 若序列 $X=\{B,C,A,D,B,C,D\}$,
 $Y=\{A,C,B,A,B,C,D\}$, 请给出序列X和Y的一个最长公共子序列_____
- $\{BABCD\}$ 或 $\{CABCD\}$

最优子结构性质

设字符串 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为：

$Z = \{z_1, z_2, \dots, z_k\}$ ，那么以下结论成立：

1. 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，且 $Z_{k-1} = \{z_1, z_2, \dots, z_{k-1}\}$ 是 $X_{m-1} = \{x_1, x_2, \dots, x_{m-1}\}$ 和 $Y_{n-1} = \{y_1, y_2, \dots, y_{n-1}\}$ 的最长公共子序列；
2. 若 $x_m \neq y_n$ 且 $x_m \neq z_k$ ，则 Z 是 X_{m-1} 和 Y 的最长公共子序列；
3. 若 $x_m \neq y_n$ 且 $y_n \neq z_k$ ，则 Z 是 X 和 Y_{n-1} 的最长公共子序列。

证明（反证法）：

若 $z_k \neq x_m$ ，则 $\{z_1, z_2, \dots, z_k, x_m\}$ 是 X 和 Y 的长度为 $k+1$ 的公共子序列。这与 Z 是 X 和 Y 的最长公共子序列**矛盾**，所以有 $z_k = x_m$ 。

同理可证， $z_k = y_n$ 。

最优子结构性质

设字符串 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为:

$Z = \{z_1, z_2, \dots, z_k\}$, 那么以下结论成立:

1. 若 $x_m = y_n$, 则 $z_k = x_m = y_n$, 且 $Z_{k-1} = \{z_1, z_2, \dots, z_{k-1}\}$ 是 $X_{m-1} = \{x_1, x_2, \dots, x_{m-1}\}$ 和 $Y_{n-1} = \{y_1, y_2, \dots, y_{n-1}\}$ 的最长公共子序列;
2. 若 $x_m \neq y_n$ 且 $x_m \neq z_k$, 则 Z 是 X_{m-1} 和 Y 的最长公共子序列;
3. 若 $x_m \neq y_n$ 且 $y_n \neq z_k$, 则 Z 是 X 和 Y_{n-1} 的最长公共子序列。

证明 (反证法) :

因为 $z_k = x_m = y_n$, 易得 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的公共子序列。

如果 Z_{k-1} **不是** X_{m-1} 和 Y_{n-1} 的最长公共子序列, 则 X_{m-1} 和 Y_{n-1} 至少存在一个长度为 k 的公共子序列, 假设为 $Z' = \{z'_1, z'_2, \dots, z'_k\}$ 。

组合 Z' 和 z_k 可**构造**长度为 $k+1$ 的新子序列 $\{z'_1, z'_2, \dots, z'_k, z_k\}$, 而且它是 X 和 Y 的公共子序列。

与前提矛盾!

最优子结构性质

设字符串 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为:

$Z = \{z_1, z_2, \dots, z_k\}$, 那么以下结论成立:

1. 若 $x_m = y_n$, 则 $z_k = x_m = y_n$, 且 $Z_{k-1} = \{z_1, z_2, \dots, z_{k-1}\}$ 是 $X_{m-1} = \{x_1, x_2, \dots, x_{m-1}\}$ 和 $Y_{n-1} = \{y_1, y_2, \dots, y_{n-1}\}$ 的最长公共子序列;
2. 若 $x_m \neq y_n$ 且 $x_m \neq z_k$, 则 Z 是 X_{m-1} 和 Y 的最长公共子序列;
3. 若 $x_m \neq y_n$ 且 $y_n \neq z_k$, 则 Z 是 X 和 Y_{n-1} 的最长公共子序列。

证明 (反证法) :

因为 $x_m \neq y_n$ 且 $x_m \neq z_k$, 则 Z 是 X_{m-1} 和 Y 的公共子序列。

如果 X_{m-1} 和 Y 有长度大于 k 的公共子序列 Z' , 则 Z' 也是 X 和 Y 的公共子序列。

与前提矛盾!

Z 是 X_{m-1} 和 Y 的最长公共子序列

最优子结构性质

设字符串 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为:

$Z = \{z_1, z_2, \dots, z_k\}$, 那么以下结论成立:

1. 若 $x_m = y_n$, 则 $z_k = x_m = y_n$, 且 $Z_{k-1} = \{z_1, z_2, \dots, z_{k-1}\}$ 是 $X_{m-1} = \{x_1, x_2, \dots, x_{m-1}\}$ 和 $Y_{n-1} = \{y_1, y_2, \dots, y_{n-1}\}$ 的最长公共子序列;
2. 若 $x_m \neq y_n$ 且 $x_m \neq z_k$, 则 Z 是 X_{m-1} 和 Y 的最长公共子序列;
3. 若 $x_m \neq y_n$ 且 $y_n \neq z_k$, 则 Z 是 X 和 Y_{n-1} 的最长公共子序列。

证明 (反证法) :

因为 $x_m \neq y_n$ 且 $y_n \neq z_k$, 则 Z 是 X 和 Y_{n-1} 的公共子序列。

如果 X 和 Y_{n-1} 有长度大于 k 的公共子序列 Z' , 则 Z' 也是 X 和 Y 的公共子序列。

与前提矛盾!

Z 是 X 和 Y_{n-1} 的最长公共子序列

状态表示

- 输入序列对 (X_{m-1}, Y_{n-1}) , (X_{m-1}, Y_n) 和 (X_m, Y_{n-1}) 都分别表示一个子问题
- 子问题可以通过两个参数确定, 即序列 X 的长度和序列 Y 的长度
- $C(i, j)$ 表示序列 $X_i = \{x_1, x_2, \dots, x_i\}$ 和 $Y_j = \{y_1, y_2, \dots, y_j\}$ 的最长公共子序列长度
- $C(m, n)$ 则表示原问题的最长公共子序列长度

状态递推方程

◆ 当 $i = 0$ 或 $j = 0$ 时, $C(i, j) = 0$;

◆ 当 $i, j > 0$ 时, $C(i, j)$ 的求解包括两种情况:

1. $x_i = y_j$ 时, (X_{i-1}, Y_{j-1}) 的最长公共子序列末尾添加元素 $x_i (= y_j)$, 即可得到 (X_i, Y_j) 的最长公共子序列
2. $x_i \neq y_j$ 时, (X_i, Y_j) 的最长公共子序列等于 (X_i, Y_{j-1}) 和 (X_{i-1}, Y_j) 的最长公共子序列的较大者

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

状态表示和状态转移方程

□ 状态:

两个输入子序列的最长公共子序列: $X[1:i] = [x_1, x_2, \dots, x_i], Y[1:j] = [y_1, y_2, \dots, y_j]$

□ 状态表示:

$C[i, j]$ 表示输入子序列 $X[1:i]$ 和 $Y[1:j]$ 的最长公共子序列的长度

□ 状态转移方程:

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

计算最优值

- ◆ 总共有 $\Theta(nm)$ 个子问题，自底向上地求解每一个子问题 $C(i, j)$ ，并且保存在数组 $C[i][j]$ 中
 - $C[0][j]$ 和 $C[i][0]$ 的值初始化为0，此时表示有一个输入串为空；
 - 求解 $X_1 = \{x_1\}$ 与所有 $Y_j, 0 \leq j \leq n$ 的最长公共子序列长度，即 $C[1][j]$
 - 求解 $X_2 = \{x_1, x_2\}$ 与所有 $Y_j, 0 \leq j \leq n$ 的最长公共子序列长度，即 $C[2][j]$
 -
 - 求解 $X_m = \{x_1, \dots, x_m\}$ 与所有 $Y_j, 0 \leq j \leq n$ 的最长公共子序列长度，即 $C[m][j]$

◆ 怎么获取最长公共子序列？

关键是子问题之间的**关系**，用一个二维数组B记录：

- $B[i][j] = 1$ ，表示 $C[i][j]$ 的值从 $C[i-1][j-1]$ 得到， **$X[i]$ 是公共字符**
- $B[i][j] = 2$ ，表示 $C[i][j]$ 的值从 $C[i-1][j]$ 得到，**公共字符存在 X_{i-1} 和 Y_j 中**
- $B[i][j] = 3$ ，表示 $C[i][j]$ 的值从 $C[i][j-1]$ 得到，**公共字符存在 X_i 和 Y_{j-1} 中**

计算最优值

例: $X=ABCBDAB$, $Y=BDCABA$

为便于观察, 将 $b=1$ 标为斜箭头, 2标为上箭头, 3标为左箭头

$i \setminus j$	0	1 (B)	2 (D)	3 (C)	4 (A)	5 (B)	6 (A)	
0	0	0	0	0	0	0	0	
1 (A)	0	0 ↑	0 ↑	0 ↑	1 ↖	1 ←	1 ↖	
2 (B)	0	1 ↖	1 ←	1 ←	1 ↑	2 ↖	2 ←	B
3 (C)	0	1 ↑	1 ↑	2 ↖	2 ←	2 ↑	2 ↑	C
4 (B)	0	1 ↖	1 ↑	2 ↑	2 ↑	3 ↖	3 ←	B
5 (D)	0	1 ↑	2 ↖	2 ↑	2 ↑	3 ↑	3 ↑	
6 (A)	0	1 ↑	2 ↑	2 ↑	3 ↖	3 ↑	4 ↖	A
7 (B)	0	1 ↖	2 ↑	2 ↑	3 ↑	4 ↖	4 ↑	

对 b 递归推导
得解: BCBA

算法实现

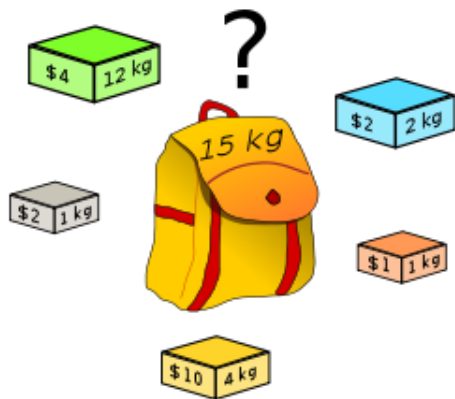
```
#define MAX 1000001
int lcsLength(char *strX, char *strY) {
    int C[MAX][MAX], B[MAX][MAX], i, j;
    int m = strlen(strX) + 1;
    int n = strlen(strY) + 1;
    for (i = 0; i < m; i++)
        C[i][0] = 0; //初始化第一行
    for (j = 0; j < n; j++)
        C[0][j] = 0; //初始化第一列
    for (i = 1; i < m; i++) {
        for (j = 1; j < n; j++) {
            if (strX[i-1] == strY[j-1]) {
                C[i][j] = C[i-1][j-1] + 1;
                B[i][j] = 1;
            } else if (C[i-1][j] >= C[i][j-1]) {
                C[i][j] = C[i-1][j];
                B[i][j] = 2;
            } else {
                C[i][j] = C[i][j-1];
                B[i][j] = 3;
            }
        }
    } // end for(j)
    } // end for(i)
    return C[m-1][n-1];
}
```

- ◆ lcsLength的时间复杂度为 $O(nm)$ ，空间复杂度也为 $O(nm)$ ，需要两个二维数组来存储最优值和最优方案
- ◆ **改进方案一**：在 $C[i-1][j-1]$ 中存储 $B[i][j]$ 的值，省略B的存储空间
- ◆ **改进方案二**：如果只要求最长公共子序列的长度，则存储空间可以缩小到 $2n$

0-1背包问题

问题描述： 给定n种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C ，问应如何选择入背包的物品，使得装入背包中物品的总价值最大？

在选择装入背包的物品时，对每种物品 i 只有**两种选择**，即**装入**背包或**不装入**背包。不能将物品装入背包多次，也不能只装入部分的物品，因此称之为**0-1背包问题**。



$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{st.} \quad & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

最优子结构性质

如果 $(x_1, x_2, \dots, x_n)$, $x_i \in \{0, 1\}$, $1 \leq i \leq n$ 是0-1背包问题的一个**最优解**, 则 $(x_1, x_2, \dots, x_{n-1})$ 是下面相应子问题的一个最优解:

$$\max \sum_{i=1}^{n-1} v_i x_i \quad \text{s.t.} \quad \begin{cases} \sum_{i=1}^{n-1} w_i x_i \leq C - w_n x_n \\ x_i \in \{0, 1\}, 1 \leq i \leq n-1 \end{cases}$$

证明 (反证法): **与前提矛盾!**

如果 $(x_1, x_2, \dots, x_{n-1})$ **不是**子问题的最优解, 则该子问题至少**存在**一个**更好**的方案, 记为 $(x'_1, x'_2, \dots, x'_{n-1})$, 满足:

$$\sum_{i=1}^{n-1} v_i x'_i > \sum_{i=1}^{n-1} v_i x_i$$

在向量 $(x'_1, x'_2, \dots, x'_{n-1})$ 后面增加一个分量 x_n **构造**原问题的一个新方案:

$$(x'_1, x'_2, \dots, x'_{n-1}, x_n)$$

$$\sum_{i=1}^{n-1} v_i x'_i + v_n x_n > \sum_{i=1}^{n-1} v_i x_i + v_n x_n$$

$$\text{s.t.} \quad \sum_{i=1}^{n-1} w_i x_i + w_n x_n \leq C$$

状态表示和转移方程

□ 子问题可由**两个参数**确定：

✓ **待考虑装包的物品集**

✓ **背包可用容量**

□ 将所有物品按照 1 至 n 标号，待考虑装包的物品集可以用物品集中**末尾物品的标号**来描述，则得状态表示 $Val(i, p)$, $1 \leq i \leq n, 0 \leq p \leq C$ 表示在**背包可用容量**为 p ，**待考虑装包的物品集**为 $\{1, 2, \dots, i\}$ 时的最大装入物品价值。

□ $Val(n, C)$ 则表示**原问题**的最优解。

状态表示和转移方程

$Val(i, p)$, $1 \leq i \leq n$, $0 \leq p \leq C$ 表示在背包可用容量为 p , 待考虑装包的物品集为 $\{1, 2, \dots, i\}$ 时的最大装入物品价值。

$$\boxed{i = 1:} \quad Val(1, p) = \begin{cases} 0, & 0 \leq p < w_1 \\ v_1, & p \geq w_1 \end{cases}$$

$$\boxed{i > 1:} \quad Val(i, p) = \begin{cases} Val(i-1, p), & 0 \leq p < w_i \\ \max(Val(i-1, p), Val(i-1, p - w_i) + v_i), & p \geq w_i \end{cases}$$

状态表示和状态转移方程

□ 状态:

背包容量为 p 时, 还需要考虑装入物品集合 $\{1, 2, \dots, i\}$

□ 状态表示:

$Val(i, p), 1 \leq i \leq n, 0 \leq p \leq C$ 表示在**背包可用容量为 p** , **待考虑装包的物品集为 $\{1, 2, \dots, i\}$** 时的最大装入物品价值。

□ 状态转移方程:

$$Val(1, p) = \begin{cases} 0, & 0 \leq p < w_1 \\ v_1, & p \geq w_1 \end{cases}$$

$$Val(i, p) = \begin{cases} Val(i-1, p), & 0 \leq p < w_i \\ \max(Val(i-1, p), Val(i-1, p - w_i) + v_i), & p \geq w_i \end{cases}$$

计算最优值

- ❑ 0-1背包问题包含 $\Theta(nC)$ 个子问题，采用**自底向上**（**由易至难**）的方法求最优值，最优值保存在**二维数组** Val 中
- ❑ 最开始的时候根据边界条件计算Val中**第一行**的值（即只考虑装第1个物品的情形），它是**最简单的**子问题。
- ❑ 然后，根据递推公式**依次**计算 Val
 - 第 2 行
 - 第 3 行
 -
 - 第 n 行



计算最优值

0-1背包问题包含 $\Theta(nC)$ 个子问题，采用自底向上（由易至难）的方法求最优值，最优值保存在二维数组Val中。

例： $n = 5$, $C = 10$, $w = \{2, 2, 6, 5, 4\}$, $v = \{6, 3, 5, 4, 6\}$

i \ p	0	1	2	3	4	5	6	7	8	9	10
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	9	9	9	9	9	9	9
3	0	0	6	6	9	9	9	9	11	11	14
4	0	0	6	6	9	9	9	10	11	13	14
5	0	0	6	6	9	9	12	12	15	15	15

算法实现

```
double binaryKnapsack(int numItems,int *w,double *v,int capacity){  
    int i, j;  
    double Val[MaxN][MaxC];  
    memset(Val,0,sizeof(Val));  
    for(i=0; i<numItems; i++)  
        for(j=0; j<=capacity; j++){  
            Val[i][j] = Val[i-1][j];  
            if(j >= w[i] && Val[i-1][j] < Val[i-1][j-w[i]]+v[i] )  
                Val[i][j]=Val[i][j-w[i]]+v[i] ;  
        }  
    return Val[numItems-1][capacity];  
}
```

计算 $val(i, p)$ 的值只需要Val数组中第 $i - 1$ 行中第 p 个分量之前的数值。如果计算 $val(i, p)$ 时按照 p 的值从大到小计算，那么只需要Val数组的一行即可。

实现代码

```
double binaryKnapsack(int numItems,int *w,double *v,int capacity){  
    int i,j;  
    double Val[MaxC];  
    memset(Val,0,sizeof(Val));  
    for(i=0;i<numItems;i++)  
        for(j=capacity;j>=0;j--)  
            if(j>=w[i] && Val[j]<Val[j-w[i]]+v[i])  
                Val[j]=Val[j-w[i]]+v[i];  
    return Val[capacity];  
}
```

$O(nC)$

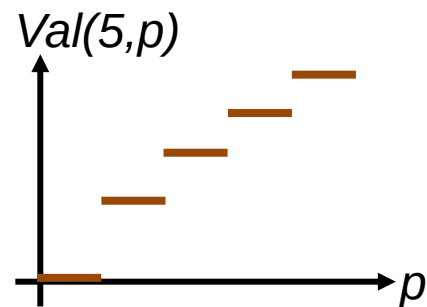
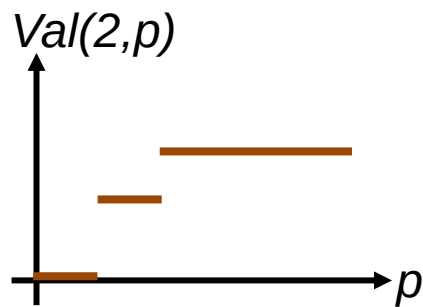
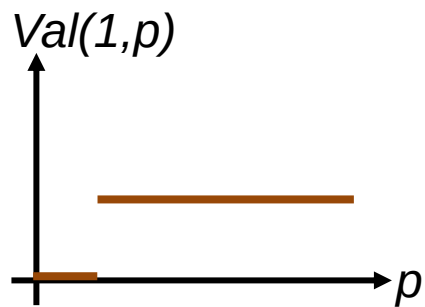
两个缺点:

1. 算法要求所给物品的重量是整数
2. 当背包容量**C很大**时, 算法需要的时间和空间都比较大

算法改进

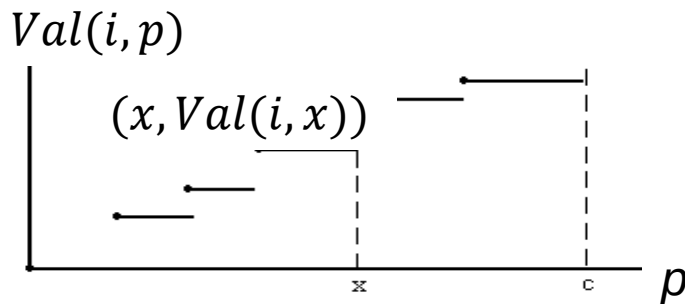
例: $n = 5$, $C = 10$, $w = \{2, 2, 6, 5, 4\}$, $v = \{6, 3, 5, 4, 6\}$

$i \setminus p$	0	1	2	3	4	5	6	7	8	9	10
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	9	9	9	9	9	9	9
3	0	0	6	6	9	9	9	9	11	11	14
4	0	0	6	6	9	9	9	10	11	13	14
5	0	0	6	6	9	9	12	12	15	15	15



跳跃点定义

- 由 $val(i, p)$ 的递归公式容易证明，在一般情况下，对每一个确定的 $i, 1 \leq i \leq n$ ，函数 $val(i, p)$ 是关于变量 p 的**阶梯状单调不减函数**。
- **跳跃点**是指在函数 $val(i, p)$ 中函数值发生跃迁的点，可以用一个实数序偶 $(x, val(i, x))$ 描述。跳跃点是阶梯函数的**描述特征**，函数 $val(i, p)$ 可由其全部跳跃点惟一确定。

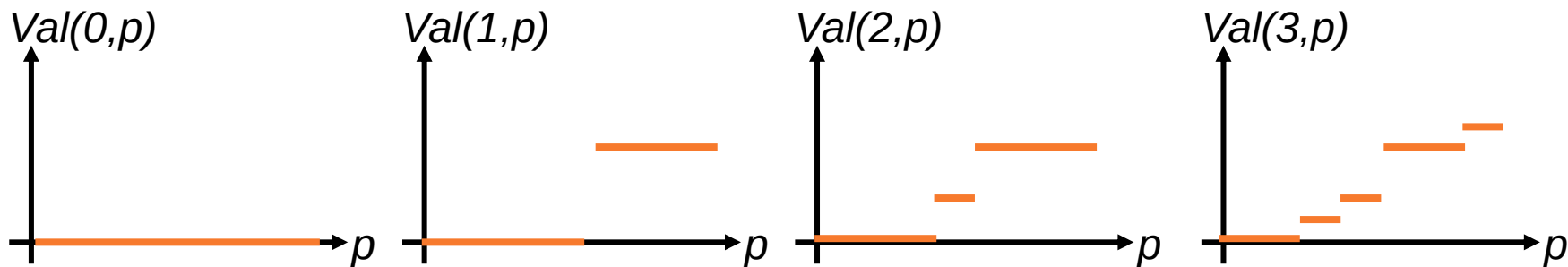


- 对每一个确定的 $i, 1 \leq i \leq n$ ，用一个表 $p[i]$ 存储函数 $val(i, p)$ 的全部跳跃点；类似地，整个状态矩阵 $val(i, p)$ 则可用跳跃点集合来表示。

跳跃点示例

例: $n = 3$, $c = 6$, $w = \{4, 3, 2\}$, $v = \{5, 2, 1\}$

- ◆ $p[0] = \{(0, 0)\}$
- ◆ $p[1] = \{(0, 0); (4, 5)\}$
- ◆ $p[2] = \{(0, 0); (3, 2); (4, 5)\}$
- ◆ $p[3] = \{(0, 0); (2, 1); (3, 2); (4, 5); (6, 6)\}$



跳跃点的递归求解

函数 $val(i, p)$ 的递归定义:

$$val(i, p) = \max(val(i-1, p), val(i-1, p - w_i) + v_i)$$

$val(i, p)$ 的跳跃点集合 $p(i)$ 是否可以由 $p(i-1)$ 构造啦?

- ① 先由 $p[i-1]$ 计算出 $q[i-1]$
- ② 合并 $p[i-1]$ 和 $q[i-1]$, 并清除其中的受控跳跃点
- ③ $p[0] = \{(0, 0)\}$

- ① 跳跃点集 $p(i-1)$: 由函数 $val(i-1, p)$ 决定, 对应于**不装第 i 个物品的所有装包情形**
- ② 跳跃点集 $q(i-1)$: 由函数 $val(i-1, p - w_i) + v_i$ 决定, 对应于**装上第 i 个物品的所有情形**, 即 $q[i-1] = p[i-1] \oplus (w_i, v_i)$
$$= \{(p + w_i, val(i-1, p) + v_i) | (p, val(i-1, p)) \in p[i-1], p + w_i \leq C\}$$
- ③ 受控跳跃点: 设 (x_1, v_1) 和 (x_2, v_2) 是 $p[i-1] \cup q[i-1]$ 中的2个跳跃点, 则**当 $x_2 \geq x_1$ 且 $v_2 < v_1$ 时**, (x_2, v_2) **受控于** (x_1, v_1) , 从而 (x_2, v_2) **不是** $p[i]$ 中的跳跃点。**除受控跳跃点外**, $p[i-1] \cup q[i-1]$ 中的其他跳跃点均为 $p[i]$ 中的跳跃点。

跳跃点计算示例

例: $n = 5$, $c = 10$, $w = \{2, 2, 6, 5, 4\}$, $v = \{6, 3, 5, 4, 6\}$

$$p[0] = \{(0, 0)\}$$

$$q[0] = p[0] \oplus (w_1, v_1) = \{(2, 6)\}$$

$$p[1] = \{(0, 0); (2, 6)\}$$

$$q[1] = p[1] \oplus (w_2, v_2) = \{(2, 3), (4, 9)\}$$

$p[2] = p[1] \cup q[1] = \{(0, 0), (2, 3), (2, 6), (4, 9)\}$, 其中跳跃点(2,3)受控于跳跃点(2,6), 将**受控跳跃点(2,3)**清除

$$p[2] = \{(0, 0), (2, 6), (4, 9)\}$$

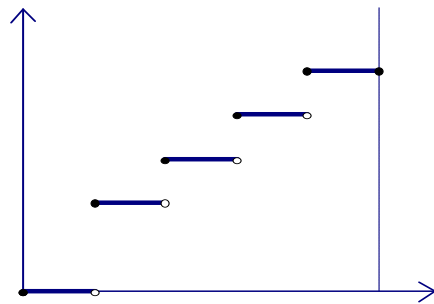
$$q[2] = p[2] \oplus (6, 5) = \{(6, 5); (8, 11); (10, 14)\}$$

$$p[3] = \{(0, 0); (2, 6); (4, 9); (8, 11); (10, 14)\}$$

$$q[3] = p[3] \oplus (5, 4) = \{(5, 4); (7, 10); (9, 13)\}$$

$$p[4] = \{(0, 0); (2, 6); (4, 9); (7, 10); (8, 11); (9, 13); (10, 14)\}$$

$$q[4] = p[4] \oplus (4, 6) = \{(4, 6); (6, 12); (8, 15)\}$$



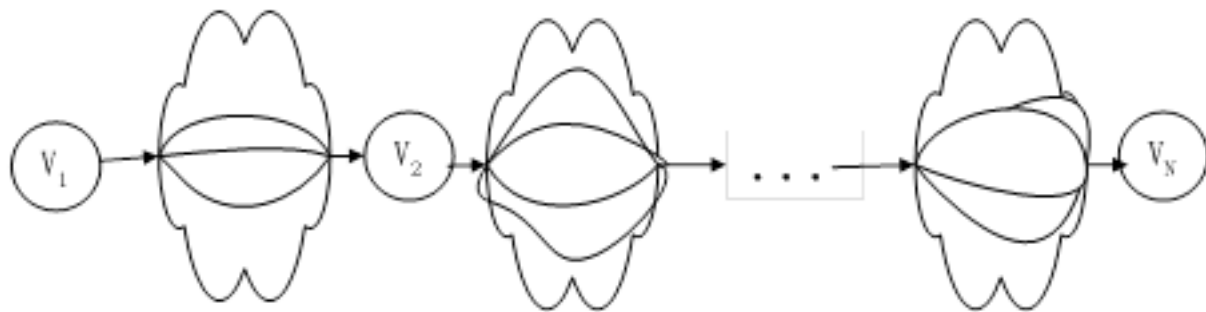
$$p[5] = \{(0, 0); (2, 6); (4, 9); (6, 12); (8, 15)\}$$

练习题：

- 1、动态规划的两个核心性质是_____和_____。
- 2、在斐波那契数列的递归解法中，直接递归的时间复杂度是_____而动态规划可优化为_____。
- 3、应用动态规划求解0-1背包问题， $val(i, p)$ 状态转移方程为_____。

第三章 小结

□ 动态规划基本原理

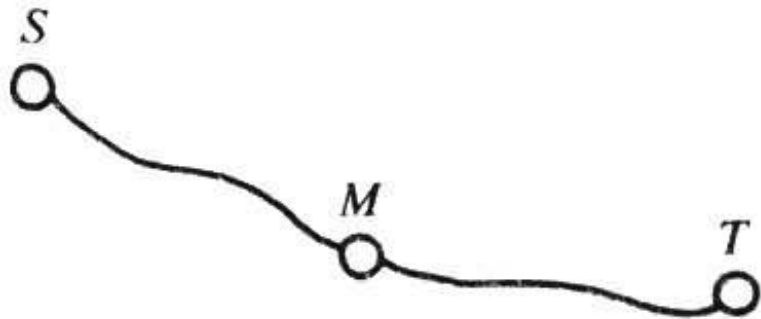


- 设计多阶段决策过程
- 分析最优子结构性质
- 建立状态表示和状态转移方程

小结

□ 最优子结构性质

问题的最优解包含其子问题的最优解。也就是说，如果把问题的最优解分解（比如划分为两个或者多个部分，或者删除第一个或者最后一个分量），得到一个子解，那么这个**子解是其相应子问题的最优解。**



最优性原理示意图

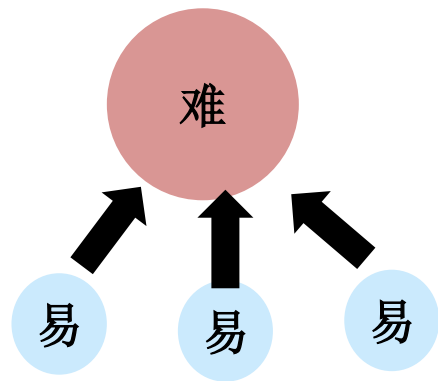
证明方法（反证法）：

- 1) 假设子解不是子问题最优解
- 2) 基于子解**构造**原问题的一个新解
- 3) 证明**新解**是原问题的**更好答案**，与**前提矛盾**！

小结

□ 最优值计算

- 状态表示和状态转移方程确定问题解之间的**递推关系**
- **有限个**（多项式级别）状态/子问题
- 自底向上（**由易至难**）求解各状态值



构造状态转移方程：

- 1) 原问题最优解**划分**为多个子解：矩阵连乘 最长公共子序列
- 2) 原问题最优解**约简**为一个子解：多段图最短路径、0-1背包

第三章 作业

□ P83 3-3 3-4

谢谢 THANKS